



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

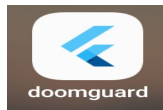
KJ SOMAIYA SCHOOL OF ENGINEERING

Department of Information Technology

Artificial Intelligence and Data Science (AIDS)

INTERNSHIP PROJECT REPORT

Doom Guard — Digital Wellbeing Application



A Flutter-based Screen Time Management and Focus Productivity App

Submitted By	Bijet Nayek (16014224081), Vini Pandhare (16014224085), Abhinay Pande (16014224084), Eshita Patil (16014224089)
Programme	Second Year, B.Tech — Artificial Intelligence and Data Science
Department	Information Technology (IT)
Faculty Guide	Assistant Professor Deepti Patole
Academic Year	2025 – 2026
Internship Duration	20th February 2026 – 14th July 2026

Abstract

Doom Guard is a Flutter-based Android application developed as part of the internship project under the guidance of Assistant Professor Deepti Patole, Department of Information Technology, KJ Somaiya School of Engineering. The application addresses the growing problem of excessive smartphone usage and doomscrolling — the compulsive habit of endlessly scrolling through social media and news feeds — which has been linked to reduced productivity, poor mental health, and disrupted sleep patterns.

The application provides users with real-time screen time monitoring, a Pomodoro-based focus timer, streak and badge-based gamification, smart notification nudges, strict mode with downtime enforcement, and cloud-synced user progress through Firebase and Cloud Firestore. The system was developed using the Flutter framework with Dart, following a Provider-based state management architecture, and tested on a physical Android device running Android 16 (API 36).

The internship spanned eight weeks from 20th February 2026 to 14th July 2026, covering the full software development lifecycle — from requirements gathering and UI design to backend integration, gamification logic, notification systems, and final deployment preparation.

Table of Contents

- 1. Introduction**
- 2. Objectives**
- 3. Technology Stack**
- 4. System Architecture**
- 5. Module-wise Description**
 - 5.1 Authentication and User Management**
 - 5.2 Screen Time Tracking**
 - 5.3 Focus Timer (Pomodoro)**
 - 5.4 Streak System**
 - 5.5 Badge and Achievement System**
 - 5.6 Smart Nudges and Notifications**
 - 5.7 Settings and Strict Mode**
 - 5.8 Privacy Policy and Onboarding**
- 6. Week-wise Work Log**
- 7. Challenges and Solutions**
- 8. Results and Testing**
- 9. Conclusion and Future Scope**
- 10. References**

1. Introduction

The modern smartphone has become an inseparable part of daily life, yet excessive and unregulated usage has given rise to a phenomenon popularly known as "doomscrolling" — the act of spending an extended period of time consuming large quantities of negative online content. Studies in behavioural psychology suggest that prolonged social media usage disrupts sleep, reduces attention span, and contributes to anxiety and reduced productivity, particularly among young adults and students.

DoomGuard was conceived and developed as a solution to this problem. It is a digital wellbeing Android application designed to help users monitor their screen time, set usage goals, build consistent focus habits, and receive intelligent nudges when usage patterns become excessive. The application was developed by a team of four second-year B.Tech students from the Artificial Intelligence and Data Science programme at KJ Somaiya School of Engineering, under the mentorship of Assistant Professor Deepti Patole, Department of Information Technology.

Unlike passive screen time tracking apps, DoomGuard combines real-time usage analytics with a gamified rewards system, making the process of building healthy screen habits engaging rather than restrictive. The app tracks which applications the user spends the most time on, sends personalised notifications when limits are approached or exceeded, and rewards users for maintaining consistent focus sessions and daily streaks through a badge-based achievement system.

The project was executed over eight weeks, following an incremental development approach where each week addressed a distinct functional area of the application — from initial Flutter learning and UI scaffolding to Firebase integration, gamification logic, and final testing on a physical device.

2. Objectives

The primary objectives of the DoomGuard internship project are as follows:

- To develop a fully functional Android application using the Flutter framework and Dart programming language that monitors and displays real-time screen time usage.
- To implement Google Sign-In authentication using Firebase Authentication and persist user data across sessions via Cloud Firestore.
- To build a Pomodoro-based focus timer that records completed sessions and cumulative focus minutes on a daily basis.
- To design a streak tracking system that rewards users for maintaining consistent daily activity and building healthy usage habits.
- To develop a badge and achievement system that automatically unlocks badges based on user statistics, providing gamified motivation.
- To implement a smart nudge notification system that monitors per-app daily usage, continuous single-app sessions, and total screen time, and delivers tiered notifications when thresholds are reached.
- To provide a Strict Mode feature that enforces usage restrictions during a user-defined wind-down period.
- To integrate a Privacy Policy screen with a one-time acceptance gate, ensuring informed user consent upon first launch.
- To synchronise all user progress data with Cloud Firestore, enabling persistence across devices and app reinstalls.
- To test the complete application on a physical Android device (OnePlus CPH2585, Android 16 / API 36) and verify functionality across all modules.

3. Technology Stack

The following technologies and libraries were employed in the development of DoomGuard:

Technology / Library	Version	Purpose
Flutter	3.x	Cross-platform mobile UI framework
Dart	3.3+	Primary programming language
Firebase Authentication	^5.3.1	Google Sign-In and user session management
Cloud Firestore	^5.4.4	NoSQL cloud database for user data persistence
Provider	^6.1.1	State management across the application
GoRouter	^13.2.0	Declarative navigation with auth-aware redirects
SharedPreferences	^2.2.2	Local key-value storage for settings and daily stats
usage_stats	^1.3.1	Android UsageStatsManager API access
flutter_local_notifications	^17.0.0	Local push notifications for nudges and alerts
fl_chart	^0.67.0	Chart and graph rendering for statistics screen
Google Sign-In	^6.2.1	OAuth 2.0 Google account authentication
google_fonts	^6.1.0	Custom typography across the app

4. System Architecture

DoomGuard follows a layered feature-first architecture that separates concerns across features, shared services, and core utilities. The application adopts the Provider pattern for reactive state management, where each major functional domain — authentication, usage tracking, timer, streaks, and settings — is encapsulated in a dedicated `ChangeNotifier` class.

4.1 Project Structure

lib/

- `core/` — Theme definitions (`AppTheme`), navigation (`GoRouter`, `createRouter`)
- `features/` — Screen-level UI for auth, dashboard, timer, stats, streaks, settings, survey, privacy
- `shared/models/` — `ChangeNotifier` state classes: `AuthState`, `UsageState`, `TimerState`, `StreakState`, `SettingsState`, `PrivacyState`
- `shared/services/` — Business logic and data services: `RealUsageService`, `LimitWatcherService`, `NotificationService`, `UserStatsService`, `AuthService`
- `shared/widgets/` — Reusable UI components

4.2 State Management

All state classes extend `ChangeNotifier` and are registered using `MultiProvider` in `main.dart`. The `_StatsSyncBootstrapper` widget sits below the `MultiProvider` and watches `AuthState` — when a user signs in, it propagates the authenticated uid to `StreakState` and `TimerState`, triggering a one-time Firestore hydration that seeds local state from cloud data on first launch or new device.

4.3 Navigation

Navigation is handled by GoRouter with an async redirect function that enforces the following flow:

- Privacy Policy not accepted → /privacy
- Not authenticated → /login
- Authenticated, survey incomplete → /survey
- Authenticated, survey complete → / (Dashboard)

The router listens to AuthState via refreshListenable and re-evaluates redirects on every auth state change.

4.4 Data Persistence

The application uses a dual-persistence strategy:

- SharedPreferences — daily counters (sessions, focus minutes), streak data, settings, and privacy acceptance flag. Used as the primary local store for fast reads.
- Cloud Firestore — users/{uid} document stores currentStreak, longestStreak, totalSavedHours, sessionsToday, focusMinutesToday, and statsUpdatedAt. Written asynchronously after every local save via fire-and-forget unawaited() calls.

On first sign-in on a new device, Firestore values are fetched and used to seed local SharedPreferences only where remote values are higher than local, ensuring correct cross-device hydration without overwriting more recent local progress.

5. Module-wise Description

5.1 Authentication and User Management

Authentication is implemented using Firebase Authentication with Google Sign-In. On first login, AuthService creates a `users/{uid}` document in Cloud Firestore containing the user's name, email, photo URL, creation timestamp, and a `surveyCompleted` flag. On subsequent logins, the document is updated with the latest login timestamp. AuthState listens to Firebase's `authStateChanges()` stream and maintains an `AuthStatus` enum (`unknown`, `authenticated`, `unauthenticated`) that drives the GoRouter redirect logic.

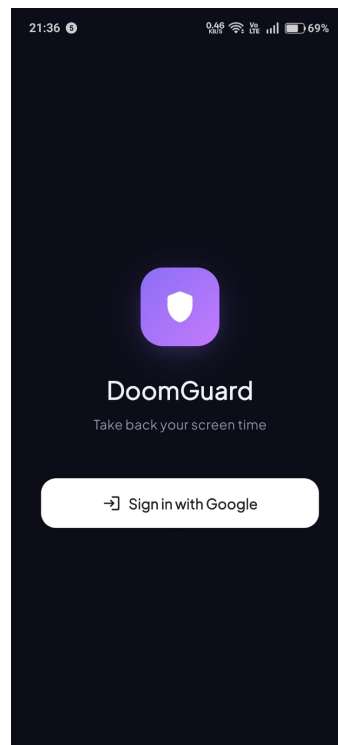


Figure 1: Google Sign-In screen

5.2 Screen Time Tracking

Screen time data is retrieved using Android's `UsageStatsManager` API via the `usage_stats` Flutter package. `RealUsageService` implements two complementary methods:

- Event-based tracking — queries `ACTIVITY_RESUMED` (type 1) and `MOVE_TO_BACKGROUND` (type 2) events from the start of the current day,

accumulating foreground duration per package. This is the primary and most accurate method.

- Aggregate API fallback — uses `queryAndAggregateUsageStats` when event-based data returns empty, which occurs on certain device configurations.

System packages (`com.android.*`, `com.google.android.gms`, etc.) are filtered out using a curated exclusion list. The service also provides a `getCurrentContinuousSession()` method that identifies whether the user has been on a single app continuously without switching, used for continuous-session nudges.

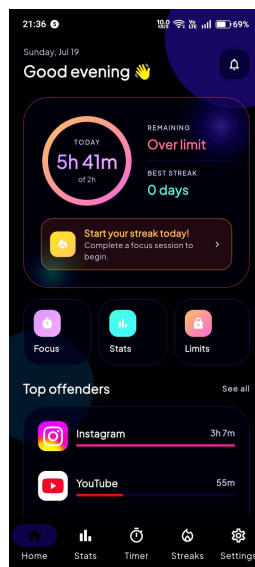


Figure 2: Home dashboard showing today's screen time and top offenders

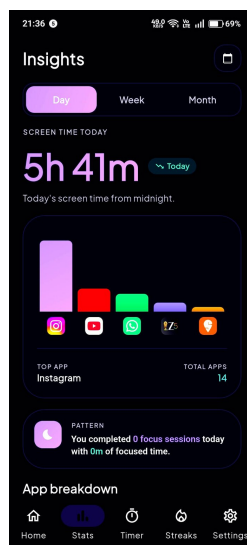


Figure 3: Insights screen with daily usage chart and app breakdown

5.3 Focus Timer (Pomodoro)

TimerState implements a Pomodoro timer with three modes: 25-minute focus sessions, 5-minute short breaks, and 15-minute long breaks. Completed focus sessions are tracked locally via SharedPreferences with daily reset logic (date string comparison). After every completed session, sessionsToday and focusMinutesToday are synced to Firestore asynchronously. The timer auto-switches to break mode on completion and rotates to a long break every four sessions.

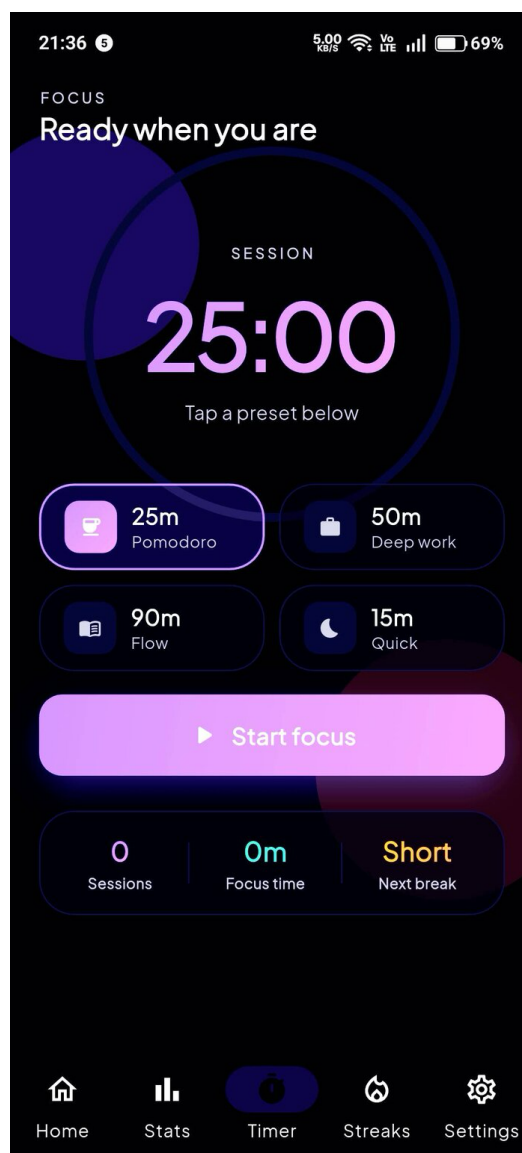


Figure 4: Focus timer with Pomodoro/Deep Work/Flow/Quick presets

5.4 Streak System

StreakState maintains currentStreak, longestStreak, totalSavedHours, and a 7-day weekly activity grid. The streak logic computes the difference in days between the last active date and today: a difference of exactly one day extends the streak, a difference greater than one resets it to zero. The streak is automatically broken if the user misses a day. All streak data is persisted locally and synced to Firestore on every update.

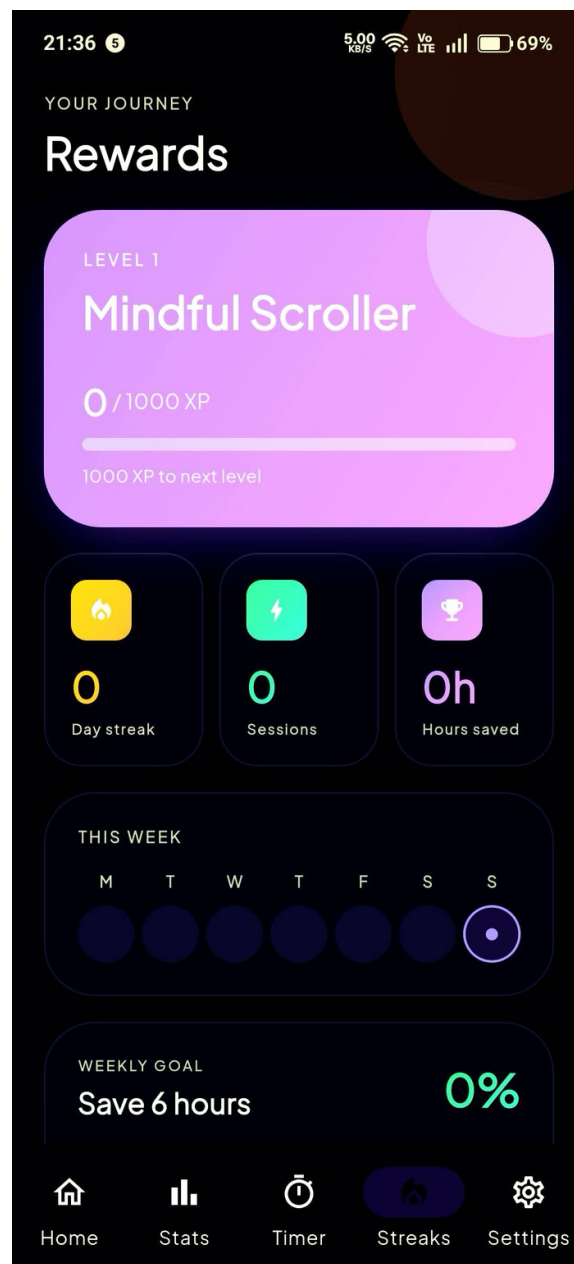


Figure 5: Streaks and Rewards screen with weekly progress

5.5 Badge and Achievement System

Badges are computed dynamically by the `computeBadges()` function, which takes five statistics as input and evaluates each badge's earned condition. The eight badges and their requirements are:

- First Step — 1 completed session or 1 saved hour
- 7-Day Streak — current streak ≥ 7 days
- Focus Master — 60+ focus minutes in a single day
- Night Owl Tamed — longest streak ≥ 3 days
- Zen Mode — current streak ≥ 14 days
- Deep Worker — 4+ focus sessions in a single day
- Time Saver — 10+ total hours saved
- 30-Day Legend — current streak ≥ 30 days

Badge computation runs at app start using values hydrated from Firestore, ensuring badges are correctly displayed even after reinstallation.

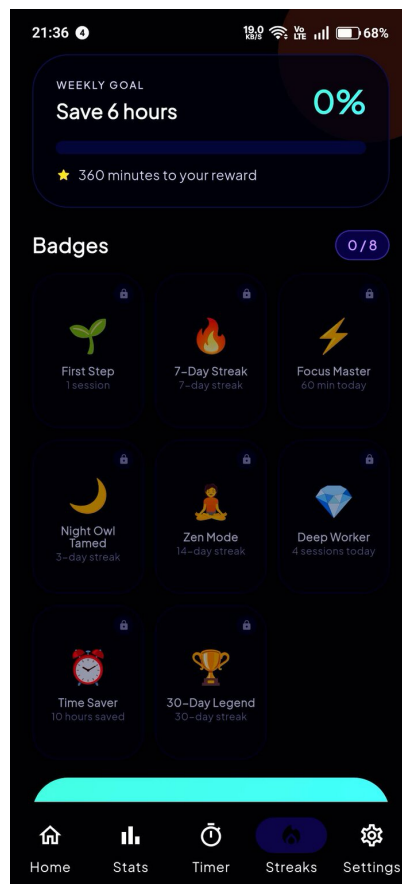


Figure 6: Badge collection screen (0/8 unlocked)

5.6 Smart Nudges and Notifications

LimitWatcherService runs a polling loop every minute while the application is open, executing three independent checks:

- Per-app daily limit check — compares current usage against per-app limits (e.g. 30 minutes for Instagram, TikTok, Snapchat; 60 minutes for YouTube). Fires a gentle nudge at 50% usage, a moderate warning at 80%, and a critical alert on limit breach.
- Continuous session check — detects uninterrupted usage of a single app exceeding 30 minutes and fires an escalating nudge (message intensity increases at 30, 60, and 90 minutes).
- Total screen time check — fires notifications when total daily usage crosses 2-hour, 3-hour, and 4-hour thresholds.

A cooldown mechanism (implemented via SharedPreferences timestamps) prevents repeated nudges: per-app cooldowns are 15 minutes, continuous session cooldowns are 10 minutes, and total screen time cooldowns are 30 minutes. All nudges respect the notificationsEnabled setting from SettingsState.

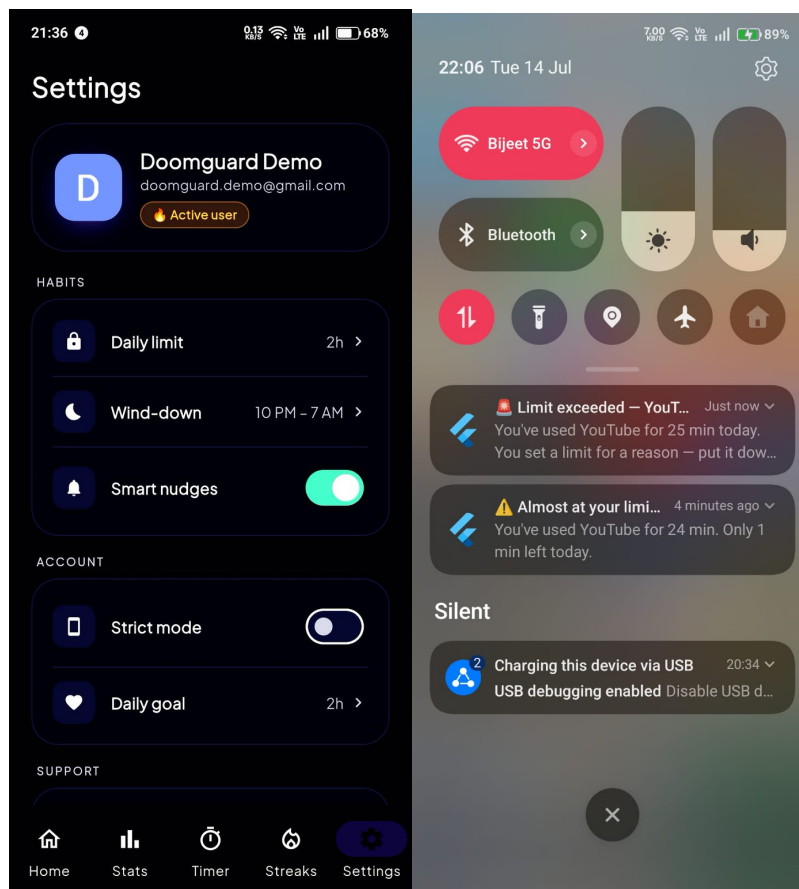


Figure 7: Settings screen with daily limit, wind-down and strict mode controls

5.7 Settings and Strict Mode

SettingsState persists the following user preferences via SharedPreferences: daily limit in minutes (default: 120), downtime start and end hours (default: 10 PM – 7 AM), notifications enabled toggle, strict mode toggle, and weekly reports preference. When strict mode is enabled, LimitWatcherService checks the current hour against the downtime window on every poll and invokes an onStrictModeBlock callback when the user is within the restricted period.

The Settings screen also supports a Family section for accounts flagged as under-13 during the onboarding survey, which allows a guardian email to be configured for weekly usage reports.

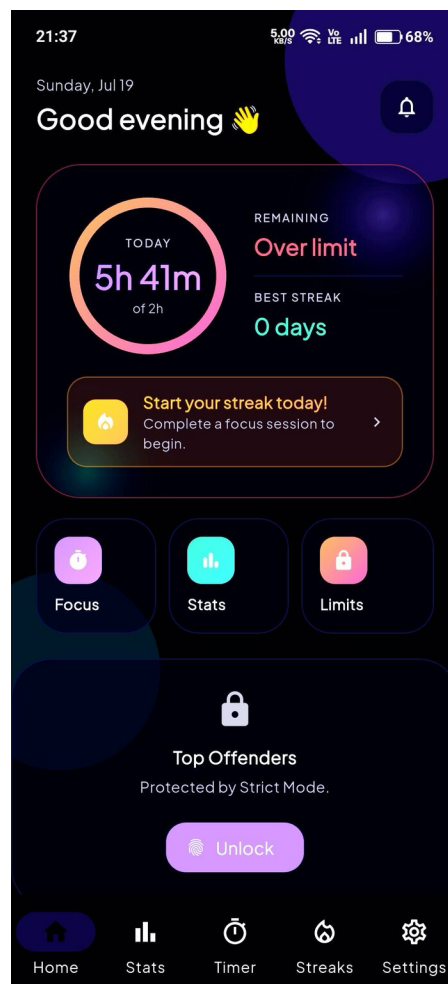


Figure 8: Home screen with Strict Mode active, locking the Top Offenders list

5.8 Privacy Policy and Onboarding

On first launch, the application displays a full Privacy Policy screen before any authentication flow. The screen presents the complete privacy policy with section headings, bullet points, and a checkbox that must be ticked before the "Continue to DoomGuard" button becomes active. Acceptance is stored via PrivacyState, which persists the `privacy_policy_accepted_v1` flag to SharedPreferences. On all subsequent launches, the GoRouter redirect bypasses the privacy screen entirely.

Following authentication, new users are directed to an onboarding survey that collects screen usage habits, age group, and primary usage goals. Survey responses are stored in the `users/{uid}` Firestore document under a survey sub-map and the `surveyCompleted` flag is set to true, preventing the survey from being shown again.

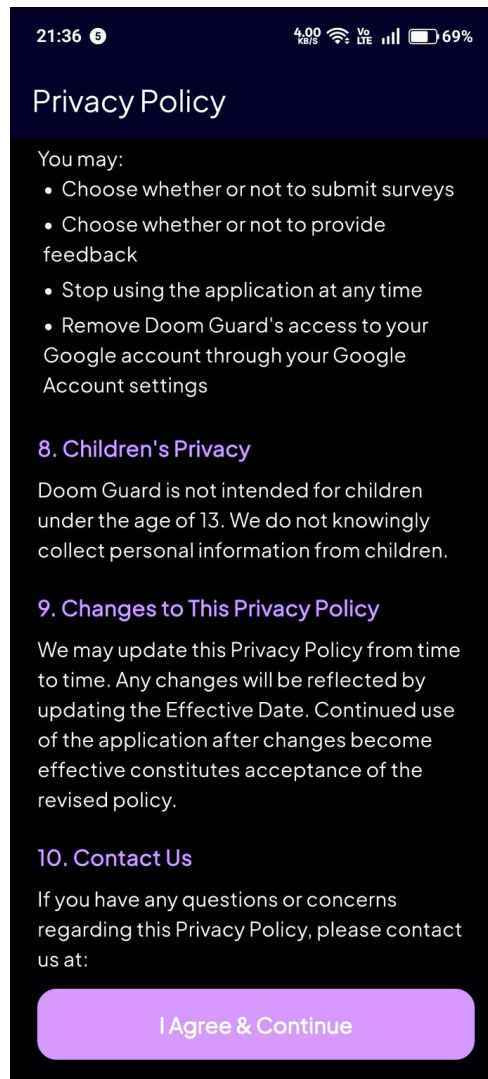


Figure 9: Privacy Policy consent screen shown on first launch

6. Week-wise Work Log

Week	Date	Task	Description	Status	Deadline
1	20th Feb 2026	Flutter Learning & Requirements	Flutter framework learning through online courses; study of Dart, widgets, state management, and project requirements gathering	Completed	27th Feb 2026
2	21st Mar 2026	Core UI and Backend Setup	Finalised feature set; built core UI screens and backend scaffold including project structure, theme, and navigation	Completed	27th Mar 2026
3	28th Mar 2026	Tracking & Reminders	Implemented usage tracking via UsageStatsManager, popup alerts, local notifications, app lock, and 10-minute usage warnings	Completed	6th Apr 2026
4	10th Apr 2026	Insights and AI	Built statistics screen with top offenders list, usage charts using fl_chart, basic AI-style usage suggestions, and user feedback collection	Completed	17th Apr 2026
5	24th Apr 2026	Firebase Integration & Auth Flow	Google Sign-In with Firebase Auth, Firestore user document creation,	Completed	30th Apr 2026

			survey flow, GoRouter redirect logic, and AuthState with ChangeNotifier		
6	20th Jun 2026	Gamification & Badge System	Badge model with computeBadges() logic, UserStats Firestore sync, cross-device hydration of StreakState and TimerState from Firestore on new device	Completed	28th Jun 2026
7	1st Jul 2026	Smart Nudges & Strict Mode	LimitWatcherService with per-app limits, continuous session detection, tiered notifications (3 severity levels), cooldown system, and downtime strict mode enforcement	Completed	8th Jul 2026
8	9th Jul 2026	Privacy Policy, Polish & Testing	Privacy Policy screen with one-time agreement gate, app router finalisation, UI polish, end-to-end testing on OnePlus CPH2585 (Android API 36), bug fixes and final build	Completed	14th Jul 2026

7. Challenges and Solutions

Challenge 1: Screen Time Accuracy Across Android Versions

The UsageStatsManager API behaves inconsistently across Android OEM variants and versions. On some devices, event-based queries returned empty results, while on others, aggregate API data was unreliable.

Solution: A dual-method approach was implemented — event-based tracking (ACTIVITY_RESUMED / MOVE_TO_BACKGROUND event pairs) as the primary method, with automatic fallback to queryAndAggregateUsageStats, and a final fallback to queryUsageStats if aggregate data was also empty. Midnight boundary clamping was added to prevent overnight sessions from bleeding into the current day's totals.

Challenge 2: GoRouter Redirect Loop

Early iterations of the authentication flow caused infinite redirect loops when the router re-evaluated before Firestore had finished fetching the user document, resulting in a blank screen.

Solution: An isFetchingUser boolean flag was added to AuthState that holds the router in its current position during the Firestore fetch. The redirect function returns null (no redirect) while isFetching is true, preventing premature navigation decisions.

Challenge 3: workmanager Kotlin Compatibility

The workmanager Flutter package (v0.5.2) used deprecated Flutter v1 embedding APIs (ShimPluginRegistry, PluginRegistrantCallback) that were removed in newer Kotlin and Flutter versions, causing a Gradle build failure.

Solution: The workmanager dependency was removed from the project as background task scheduling was not critical to the core feature set. The LimitWatcherService polling loop was implemented as a foreground timer (Timer.periodic) that runs while the application is open, providing nudge functionality without requiring background execution.

Challenge 4: Cross-Device Data Hydration

When a user reinstalled the application or signed in on a new device, all local SharedPreferences data was empty, resulting in streaks and stats being reset to zero despite Firestore containing the correct values.

Solution: A one-time hydration mechanism was implemented in `StreakState.setUid()` and `TimerState.setUid()`. On first sign-in per session, Firestore values are fetched and compared against local values — remote values are adopted only where they are higher than local values, preventing overwrite of more recent local progress while correctly seeding a fresh install.

Challenge 5: Screen Time Inflation

Initial testing showed inflated screen time figures (e.g. 3h 45m at 12:45 AM) because the usage query window was using a 24-hour rolling window instead of the start of the current calendar day.

Solution: The query start time was fixed to `DateTime(now.year, now.month, now.day)` — midnight of the current day — and all foreground event timestamps were clamped to this boundary, ensuring that usage from the previous day does not carry forward.

8. Results and Testing

8.1 Testing Environment

- Device: OnePlus CPH2585
- Android Version: Android 16 (API Level 36)
- Flutter Version: 3.x (stable channel)
- Firebase Project: DoomGuard (active, Blaze plan)

8.2 Test Results

Feature	Test Performed	Result
Google Sign-In	New user first login	Firestore document created; survey shown
Survey flow	Complete survey, re-login	Survey bypassed on subsequent logins
Screen time tracking	Used Instagram for 20 minutes	Correctly reflected in dashboard and stats
Focus timer	Completed 1 Pomodoro session	Session count and focus minutes updated in Firestore
Streak tracking	Active on two consecutive days	Streak incremented correctly; Firestore updated
Badge system	Clear app data, re-login	Badges recomputed from Firestore values on startup
Smart nudges	Exceeded app usage limit	Tiered notifications delivered at 50%, 80%, 100%
Privacy policy gate	Fresh install, first launch	Privacy screen shown; bypassed on all subsequent launches
Strict mode	Enabled strict mode during downtime hours	Downtime enforcement callback triggered
Cross-device hydration	Clear app data, re-login	Stats and streaks restored from Firestore

9. Conclusion and Future Scope

9.1 Conclusion

DoomGuard was successfully developed as a complete, functional Android application over the course of eight weeks. The project demonstrates the practical application of cross-platform mobile development using Flutter, cloud-based authentication and data persistence using Firebase, reactive state management using the Provider pattern, and event-driven notification systems for behavioural nudging.

The application fulfils all stated objectives: it accurately tracks and displays real-time screen usage, provides a Pomodoro-based focus tool, rewards consistent behaviour through a gamified streak and badge system, delivers intelligent nudges through a tiered notification mechanism, and persists all user progress securely in Cloud Firestore with cross-device synchronisation.

The internship provided practical exposure to the complete software development lifecycle, including requirements analysis, UI/UX design, mobile application development, backend integration, testing on physical hardware, and iterative debugging. The guidance of Assistant Professor Deepti Patole, Department of Information Technology, KJ Somaiya School of Engineering, was instrumental in shaping the technical direction and ensuring timely delivery of each milestone.

9.2 Future Scope

- **Background scheduling:** Implementing robust background usage monitoring using a foreground service or AlarmManager-based approach to deliver nudges even when the application is closed.
- **Guardian dashboard:** Completing the parental monitoring feature with weekly PDF usage reports sent to the configured guardian email address.
- **App-specific limits UI:** Allowing users to set custom per-application daily limits through the settings screen rather than relying on hardcoded defaults.
- **AI-driven insights:** Integrating a lightweight on-device model or cloud-based AI service to generate personalised usage pattern insights and recommendations.
- **Play Store deployment:** Publishing the application on the Google Play Store following completion of the privacy policy, data safety declaration, and content rating requirements.
- **iOS support:** Extending the application to iOS using Screen Time API and equivalent notification frameworks.

10. References

- [1] Flutter Documentation. Google LLC. <https://docs.flutter.dev>
- [2] Firebase Documentation — Authentication and Cloud Firestore. Google LLC. <https://firebase.google.com/docs>
- [3] Android UsageStatsManager API Reference. Android Developers. <https://developer.android.com/reference/android/app/usage/UsageStatsManager>
- [4] Provider Package — Flutter State Management. <https://pub.dev/packages/provider>
- [5] GoRouter Package — Declarative Routing for Flutter. https://pub.dev/packages/go_router
- [6] flutter_local_notifications Package. https://pub.dev/packages/flutter_local_notifications
- [7] usage_stats Package. https://pub.dev/packages/usage_stats
- [8] fl_chart Package — Flutter Chart Library. https://pub.dev/packages/fl_chart
- [9] Twenge, J. M. (2017). iGen: Why Today's Super-Connected Kids Are Growing Up Less Rebellious, More Tolerant, Less Happy. Atria Books.
- [10] Kross, E., et al. (2013). Facebook Use Predicts Declines in Subjective Well-Being in Young Adults. PLOS ONE.